

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

Анализ больших текстовых данных и текстовый поиск

Лекция 6

06 октября 2025

Анализ тональности

Анализ тональности — выявление эмоциональной окраски текста — позитивной, негативной или нейтральной

Цель — понять настроение автора и отношение к объекту (товару, событию, бренду и т. д.)

Типичные применения:

- Маркетинг: анализ отзывов о продуктах, брендах
- Политика: мониторинг общественных настроений
- HR-аналитика: анализ отзывов сотрудников о компании
- Социология и СМИ: изучение реакции аудитории на события

Методы анализа тональности. Rule-based

Подход 1. Правила и словари (Rule-based / Lexicon-based)

Идея: оценить настроение текста на основе списков слов с заранее заданной эмоциональной окраской.

Словари тональности:

- AFINN — английский словарь, каждое слово имеет «оценку» от -5 до $+5$
- SentiWordNet — расширение WordNet с весами для позитивных и негативных смыслов
- RuSentiLex — русскоязычный словарь тональности

Методы анализа тональности. Rule-based

Принцип работы:

- Разбить текст на слова
- Найти их в словаре
- Суммировать оценки слов
- Определить общий знак (позитив / негатив / нейтрально)

Проблемы:

- Ирония и сарказм: «Ну да, просто гениально!»
- Контекст: «высокая температура» — плохо для пациента, но хорошо для моря
- Многозначность слов: «fire» — «огонь» или «уволить»

AFINN

```
#importing necessary libraries
from afinn import Afinn
import pandas as pd

#instantiate afinn
afn = Afinn()

#creating list sentences
news_df = ['les gens pensent aux chiens', 'i hate flowers',
           'he is kind and smart', 'we are kind to good people']

# compute scores (polarity) and labels
scores = [afn.score(article) for article in news_df]
sentiment = ['positive' if score > 0
              else 'negative' if score < 0
              else 'neutral'
              for score in scores]

# dataframe creation
df = pd.DataFrame()
df['topic'] = news_df
df['scores'] = scores
df['sentiments'] = sentiment
print(df)
```

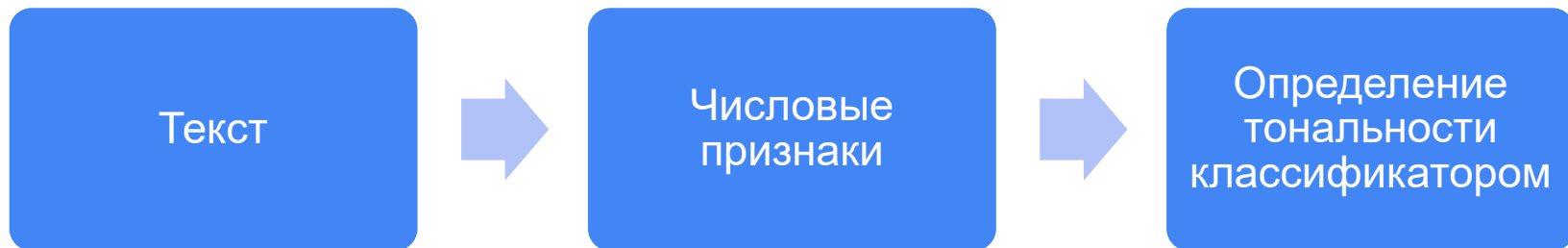


	topic	scores	sentiments
0	les gens pensent aux chiens	0.0	neutral
1	i hate flowers	-3.0	negative
2	he is kind and smart	3.0	positive
3	we are kind to good people	5.0	positive

Методы анализа тональности. ML-based

Подход 2. Машинное обучение (ML-based)

Идея: Модель учится на размеченных данных, где каждому тексту заранее известна его тональность(например: «положительный», «отрицательный», «нейтральный»)



Методы анализа тональности. ML-based

Алгоритм не знает смысла слов, но находит **статистические закономерности**:

- если в тексте часто встречаются “great”, “excellent”, “love”: вероятно, тональность положительная
- если “bad”, “terrible”, “worst”: отрицательная

Преимущества подхода:

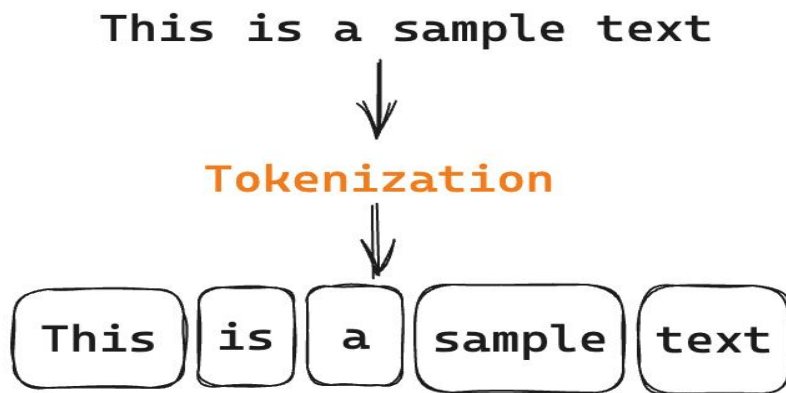
- Может адаптироваться к новой области (например, отзывы о фильмах или еде)
- Учитывает частотные закономерности, а не только словари
- Даёт количественные метрики качества (accuracy, F1-score и т. д.)

Этапы ML-based анализа тональности

Этап 1. Предобработка текста (Preprocessing)

Цель:

Подготовить текст к анализу — убрать шум и привести слова к единому стандартному виду, чтобы модель работала с осмысленными и сопоставимыми токенами



Этапы ML-based анализа тональности. Preprocessing

Основные шаги:

1. Очистка текста (Cleaning)

- Удаление знаков пунктуации, чисел, HTML-тегов, ссылок
- Приведение текста к нижнему регистру
- Удаление лишних символов, эмодзи, служебных меток

"Great!!! Delivery in 2 days 🚚" → "great delivery in days"

Этапы ML-based анализа тональности. Preprocessing

2. Токенизация (Tokenization)

Разбиение текста на минимальные смысловые единицы (токены): слова, знаки препинания, эмодзи, хэштеги и т. д.

Для чего нужна токенизация:

- Превращает сплошной текст в структурированные данные, пригодные для анализа
- Позволяет считать частоты слов, применять TF-IDF, строить эмбединги
- Является первым шагом большинства NLP-пайплайнов

"great delivery in days" → ["great", "delivery", "in", "days"]

Этапы ML-based анализа тональности. Preprocessing

3. Удаление стоп-слов (Stop words removal)

Стоп-слова — это часто встречающиеся в языке слова, которые не несут смысловой нагрузки и не помогают различать тональность

Обычно это:

- служебные части речи (предлоги, союзы, частицы)
- местоимения
- вспомогательные глаголы

["great", "delivery", "in", "days"] → ["great", "delivery", "days"]

Этапы ML-based анализа тональности. Preprocessing

4. Лемматизация / стемминг (Lemmatization / Stemming)

Лемматизация — приведение слова к его нормальной (словарной) форме, т. е. к лемме, как она представлена в словаре

Стемминг — более простая, эвристическая операция: отсечение окончаний и суффиксов без учёта грамматики

- Лемматизация понимает контекст и часть речи, поэтому “лучше” может стать “хорошо” (наречие) или “лучший” (прилагательное)
- Стемминг не различает смыслы, просто обрезает слова по правилам

Этапы ML-based анализа тональности. Preprocessing

4. Лемматизация / стемминг (Lemmatization / Stemming)

Для чего это нужно:

- Слова одной формы должны считаться одинаковыми признаками
- Уменьшает размерность данных и улучшает качество классификаторов
- Позволяет объединять формы:
 - «купил», «куплю», «купить» — «купить»
 - «cats», «catlike», «cat's» — «cat»

["great", "delivery", "days"] → ["great", "deliver", "day"]

Этапы ML-based анализа тональности. Preprocessing

5. Обработка отрицаний и эмодзи

Для чего это нужно:

Некоторые элементы текста меняют смысл целиком, хотя формально они — несущественные части речи или символы. Если их не учесть, модель ошибочно интерпретирует тональность.

- “good” — позитивное слово
- “not good” — противоположное значение

Если просто удалить «not» как стоп-слово, смысл фразы полностью теряется.

Этапы ML-based анализа тональности. Preprocessing

5. Обработка отрицаний и эмодзи

Решение:

- Связать отрицание с последующим словом, чтобы модель видела “not_good” как единый токен

Оригинал	После преобразования	Интерпретация
“not good”	“not_good”	негатив
“not bad”	“not_bad”	нейтрально-позитивное
“not interesting at all”	“not_interesting_at_all”	негатив

- Некоторые токенизаторы учитывают отрицания автоматически

Этапы ML-based анализа тональности. Preprocessing

5. Обработка отрицаний и эмодзи

Эмодзи часто заменяют слова эмоций:

“The food was 😍” = “The food was amazing.”

Решение:

Перевести эмодзи в специальные текстовые токены — они станут признаками для модели

```
text = "The movie was amazing 😍 but too long 😞"  
text = text.replace("😍", "positive_emoji").replace("😞", "negative_emoji")  
print(text)
```

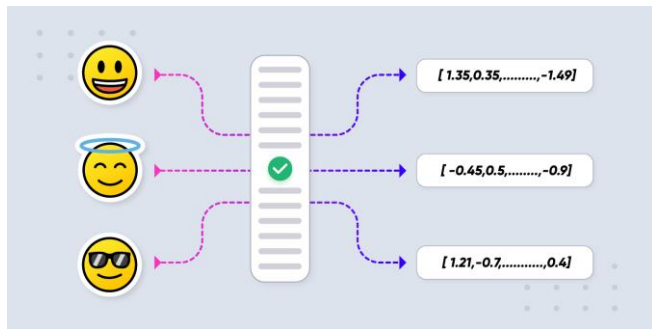
```
The movie was amazing positive_emoji but too long negative_emoji
```

Этапы ML-based анализа тональности. Preprocessing

5. Обработка отрицаний и эмодзи

Результат обработки:

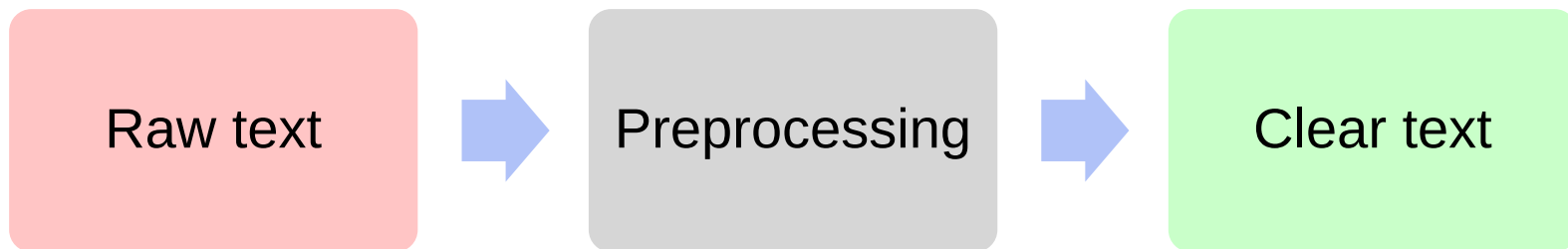
- сохраняется контекст отрицания (`not_good` $\neq$ `good`),
- добавляются эмоциональные сигналы из эмодзи,
- модель лучше различает тональность коротких сообщений и отзывов (особенно в соцсетях и чатах)



Этапы ML-based анализа тональности. Preprocessing

Результат предобработки текста

- Короткий, чистый и информативный список слов
- Без шумовых символов, стоп-слов и повторов
- Сохранён смысл (отрицания и эмоции учтены)
- Готово для этапа векторизации



Этапы ML-based анализа тональности. Preprocessing

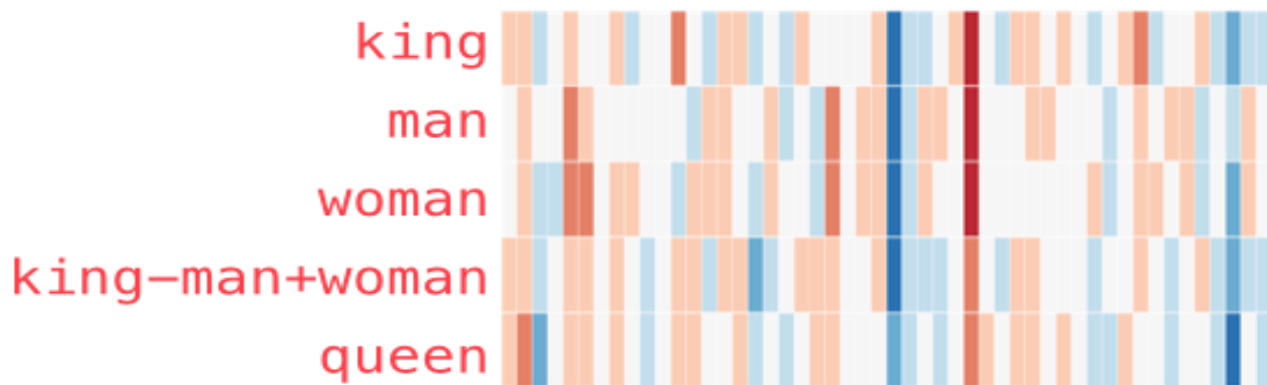
Этап	Текст / результат
Исходный текст	“The product wasn’t good 😞 but the delivery was fast!”
1. Очистка	“the product wasnt good but the delivery was fast”
2. Токенизация	["the", "product", "wasnt", "good", "but", "the", "delivery", "was", "fast"]
3. Удаление стоп-слов	["product", "wasnt", "good", "delivery", "fast"]
4. Обработка отрицаний	["not_good", "product", "delivery", "fast"]
5. Эмодзи → токен	["not_good", "product", "delivery", "fast", "negative_emoji"]
6. Лемматизация	["not_good", "product", "deliver", "fast", "negative_emoji"]

Этапы ML-based анализа тональности. Векторизация

Этап 2. Векторизация

Векторизация — это процесс преобразования текста в числовое представление (вектор), которое можно подать на вход модели машинного обучения

king - man + woman $\sim$ queen



Этапы ML-based анализа тональности. Векторизация

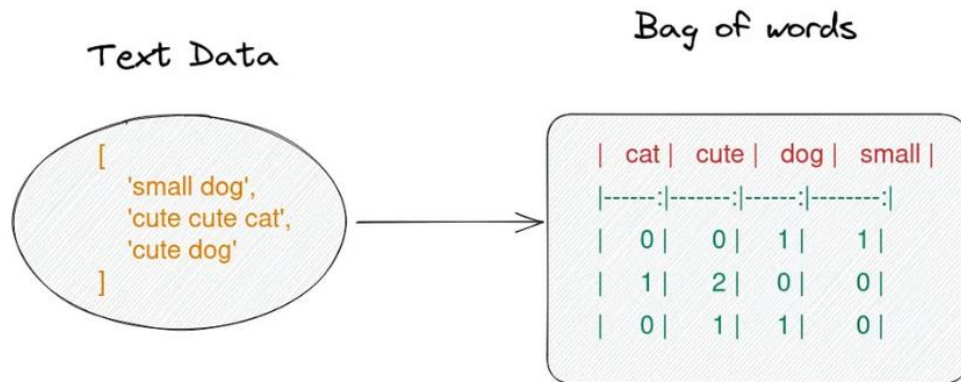
Почему это важно:

- Машинное обучение работает только с числами
- Векторизация превращает неструктурированные тексты в структурированные данные
- Позволяет:
 - сравнивать тексты между собой
 - классифицировать по тональности
 - вычислять сходство и кластеры.

Этапы ML-based анализа тональности. Векторизация

Примеры:

- Мешок слов
- TF-IDF
- Word Embeddings
 - Word2Vec
 - GloVe
 - FastText



Этапы ML-based анализа тональности. Training

Этап 3. Обучение модели (Training)

Обучение модели — это процесс, при котором алгоритм изучает зависимость между вектором признаков и меткой тональности (позитив, негатив, нейтрально) на основе размеченных примеров

Идея:

Модель смотрит на множество текстов, учится распознавать закономерности в признаках (словах, частотах, весах) и затем может предсказывать тональность нового текста

Этапы ML-based анализа тональности. Training

Классические алгоритмы машинного обучения для анализа тональности текста:

1. Naïve Bayes (Наивный байесовский классификатор)

Основная идея:

Определяем, к какому классу (позитив / негатив) относится текст, исходя из вероятности появления слов, характерных для каждого класса

Модель опирается на теорему Байеса:

$$P(\text{class} \mid \text{text}) = \frac{P(\text{text} \mid \text{class}) \cdot P(\text{class})}{P(\text{text})}$$

Этапы ML-based анализа тональности. Training

Ключевые особенности Naïve Bayes:

- Наивное (строгое) предположение: все слова в тексте считаются независимыми признаками (что, конечно, не совсем так). Это упрощает расчёты и делает метод очень быстрым
- Работает по статистике слов: считает, насколько часто слово встречается в каждом классе
- Очень эффективен для текстов, особенно с большими корпусами (тысячи и миллионы документов)
- Часто используется как базовая модель для задач анализа тональности, спама и классификации отзывов

Этапы ML-based анализа тональности. Training

Формула Байеса:

$$P(\text{class} \mid \text{text}) = \frac{P(\text{text} \mid \text{class}) \cdot P(\text{class})}{P(\text{text})}$$

Эта формула описывает, как пересчитать вероятность события “class” после того, как мы увидели текст

Компонент	Как читается	Значение
$P(\text{class} \text{text})$	«вероятность класса при данном тексте»	Что мы хотим найти: вероятность того, что текст относится, например, к positive
$P(\text{text} \text{class})$	«вероятность текста при данном классе»	Насколько типично встретить этот текст, если мы уже знаем, что он позитивный
$P(\text{class})$	«априорная вероятность класса»	Вероятность встретить текст этого класса вообще (например, 70% отзывов — позитивные)
$P(\text{text})$	«вероятность самого текста»	Какова вероятность увидеть такой текст вообще (нормирующий коэффициент)

Этапы ML-based анализа тональности. Training

Пример

$$P(\text{class} \mid \text{text}) = \frac{P(\text{text} \mid \text{class}) \cdot P(\text{class})}{P(\text{text})}$$

Пусть у нас есть только два класса: Positive и Negative.

Мы читаем текст: "The movie was great!"

Элемент	Positive	Negative
$P(\text{text} \mid \text{class})$	0.04 (в позитивных часто есть "great")	0.001
$P(\text{class})$	0.6 (60% отзывов позитивные)	0.4
$P(\text{text})$	одинакова для всех классов (можно не считать напрямую)	

$$P(\text{positive} \mid \text{text}) = \frac{0.04 \times 0.6}{P(\text{text})}$$

$$P(\text{negative} \mid \text{text}) = \frac{0.001 \times 0.4}{P(\text{text})}$$

$$0.04 \times 0.6 = 0.024 > 0.001 \times 0.4 = 0.0004$$

Текст классифицируется как **ПОЗИТИВНЫЙ**

Этапы ML-based анализа тональности. Training

Преимущества:

- Простая реализация
- Очень быстрая работа
- Хорошо масштабируется на большие корпуса
- Неплохо работает даже при сильных упрощениях

Ограничения:

- Предположение независимости слов не всегда верно
- Не учитывает порядок и контекст (“not good” может трактоваться как “good”)
- Требуется сбалансированных данных для корректных вероятностей.

Этапы ML-based анализа тональности. Training

2. SVM (Support Vector Machine). Метод опорных векторов

Основная идея:

Найти границу (гиперплоскость), которая лучше всего разделяет тексты разных классов (например, позитивные и негативные) в пространстве признаков (TF-IDF, embeddings и др.)

Как работает:

- Каждый документ представлен как вектор признаков (например, TF-IDF)
- Алгоритм ищет такую плоскость, которая максимально разделяет классы
- Опорные векторы — это точки, ближайшие к границе. Именно они определяют положение разделяющей линии.

Этапы ML-based анализа тональности. Training

SVM ищет границу, при которой зазор (margin) между классами максимален

$$\text{maximize } \frac{2}{\|w\|} \quad \text{при условии } y_i(w \cdot x_i + b) \geq 1$$

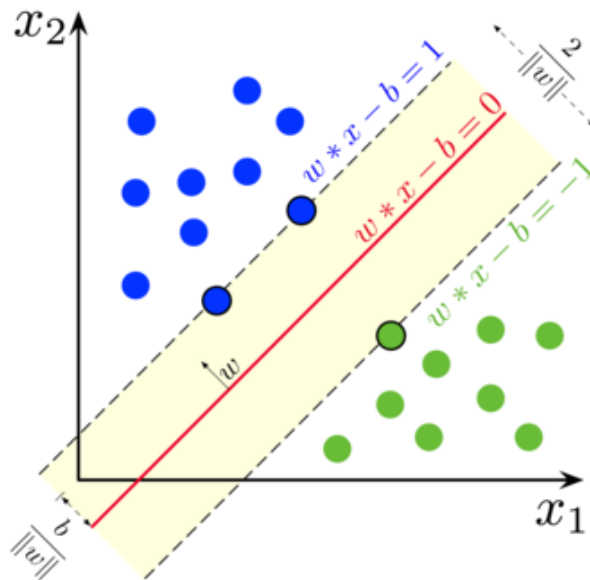
Где

w — вектор весов

b — смещение (bias)

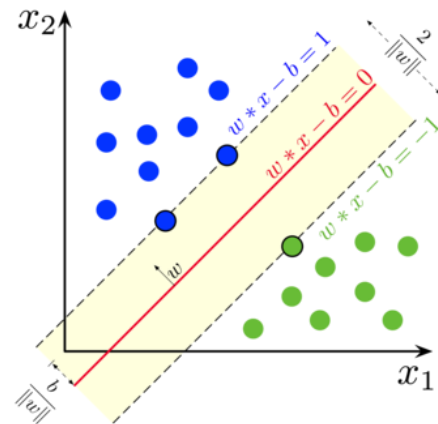
x_i — вектор документа

y_i — метка класса (+1 / -1)



Этапы ML-based анализа тональности. Training

Обозначение	Значение
x_i	вектор признаков i -го объекта (например, TF-IDF слов для текста)
y_i	метка класса: +1 (например, позитив) или -1 (негатив)
w	вектор весов — направление разделяющей гиперплоскости
b	смещение (bias), определяет положение плоскости
$w \cdot x_i + b$	скалярное произведение — показывает, на какой стороне от границы находится точка



Пусть класс положительный: $y_i = +1$

Тогда: $(+1)(w \cdot x_i + b) \geq 1 \longrightarrow w \cdot x_i + b \geq 1$

Пусть класс отрицательный: $y_i = -1$

Тогда: $(-1)(w \cdot x_i + b) \geq 1 \longrightarrow w \cdot x_i + b \leq -1$

Этапы ML-based анализа тональности. Training

Что, если у нас 3 или больше классов?

Например:

- позитив
- нейтральный
- негатив

Модель должна предсказать:

“Какова вероятность, что текст принадлежит каждому из трёх классов?”

Этапы ML-based анализа тональности. Training

SVM для многоклассовой классификации

Метод	Идея	Количество моделей
One-vs-Rest (OvR) (один против всех)	Для каждого класса обучается своя SVM-модель, отличающая этот класс от всех остальных	K
One-vs-One (OvO) (каждый против каждого)	Для каждой пары классов строится своя SVM-модель	$K(K-1)/2$
Direct Multiclass (реже)	Единая оптимизационная задача с несколькими гиперплоскостями (теоретически возможно, но вычислительно сложно)	1

Этапы ML-based анализа тональности. Training

Преимущества:

- Эффективен для высокомерных и разреженных данных — типичных для TF-IDF
- Хорошо работает даже при небольшом количестве примеров
- Устойчив к переобучению
- Может использовать нелинейные границы (через “kernel trick”)

Недостатки:

- Плохо масштабируется на очень большие наборы данных
- Не выдаёт вероятность, а только метку класса (positive / negative)
- Требует нормализации данных и подбора гиперпараметров

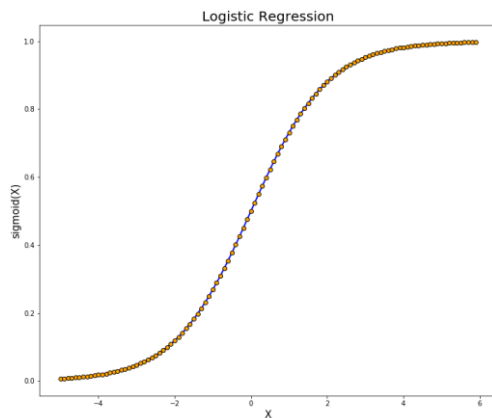
Этапы ML-based анализа тональности. Training

3. Логистическая регрессия (Logistic Regression)

Основная идея:

Логистическая регрессия строит модель вероятности того, что текст принадлежит к определённому классу (например, позитив / негатив)

Она оценивает зависимость между вектором признаков (например, TF-IDF слов) и вероятностью класса



Этапы ML-based анализа тональности. Training

Математическая основа:

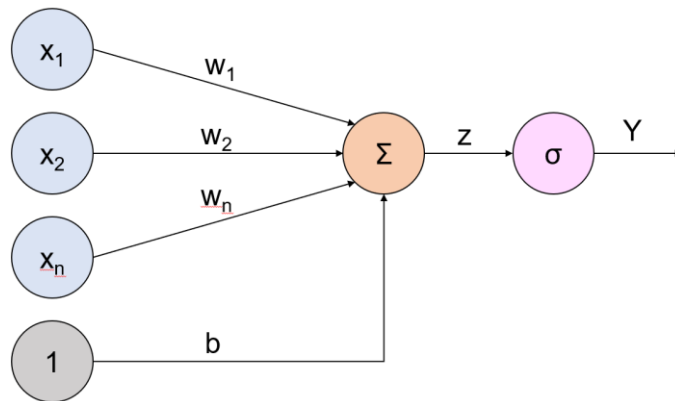
Для каждого текста x :

$$P(y = 1 \mid x) = \sigma(w \cdot x + b)$$

$\sigma(z) = \frac{1}{1+e^{-z}}$ — сигмоида, которая преобразовывает результат в диапазон от 0 до 1

w — веса (вклад каждого признака)

b — смещение (bias)



Этапы ML-based анализа тональности. Training

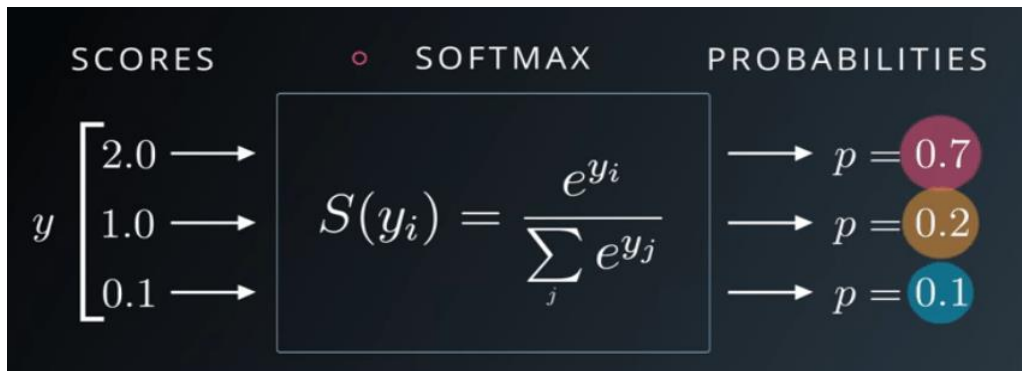
Logistic Regression для многоклассовой классификации

Для каждого класса k строится своя линейная модель:

$$z_k = w_k \cdot x + b_k$$

Потом все оценки переводятся в вероятности через Softmax-функцию:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

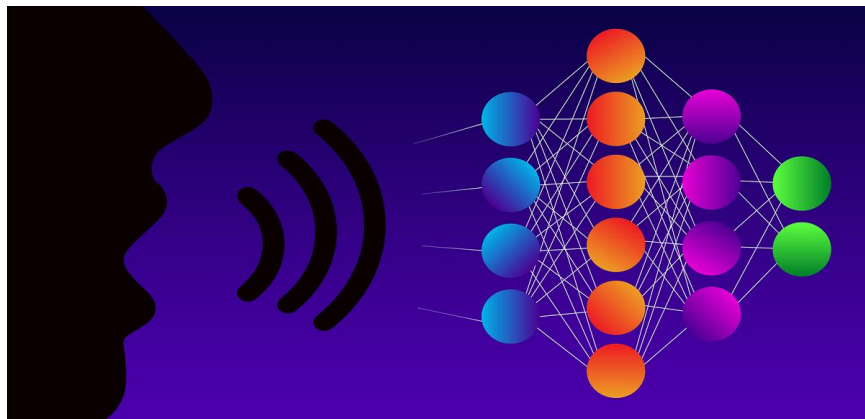


Методы анализа тональности. DL-based

Подход 3. Глубокое обучение (DL-based)

Идея: Модели глубокого обучения анализируют контекст и смысл слов в предложении, а не только их наличие или частоту.

Глубокие модели понимают, как слова связаны между собой, что особенно важно для тональности, сарказма, многозначных выражений и контекста.



Методы анализа тональности. DL-based

Модели глубокого обучения для анализа тональности

Модель	Особенность	Применение
RNN (Recurrent Neural Network)	Последовательная обработка текста, учитывает порядок слов	Простые последовательности (например, отзывы)
LSTM (Long Short-Term Memory)	Память для долгих зависимостей, не теряет контекст	Более длинные тексты, сложные фразы
BERT (Bidirectional Encoder Representations from Transformers)	Анализирует контекст в обе стороны — до и после слова	Современные системы анализа тональности и семантики

Методы анализа тональности. DL-based

Принцип работы:

- Вход: предложение превращается в последовательность токенов
- Эмбединг: слова преобразуются в векторы (Word2Vec, embeddings)
- Контекстная обработка:
 - RNN / LSTM проходят по тексту слева направо (или двунаправленно)
 - BERT учитывает контекст всех слов одновременно с помощью трансформеров (attention)
- Выход: вероятность класса (например, позитив / негатив / нейтральный)

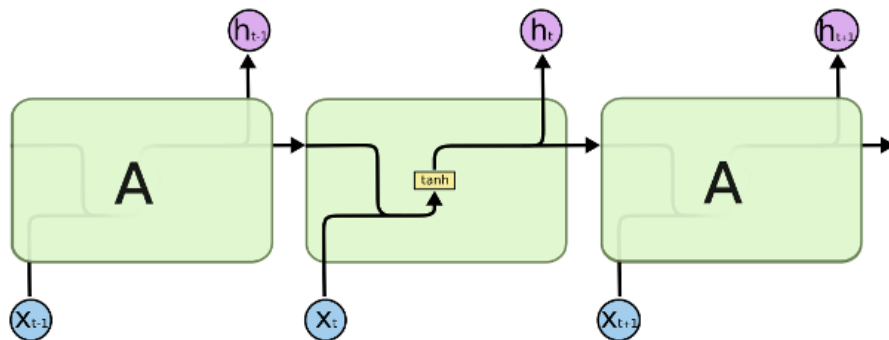
Методы анализа тональности. DL-based

RNN — Recurrent Neural Network (Рекуррентная нейронная сеть)

Идея:

В отличие от обычных нейронных сетей, RNN умеет помнить контекст —она обрабатывает последовательности данных (например, слова в тексте) с учётом того, что было раньше

RNN — это модель, где прошлое состояние влияет на текущее решение



Методы анализа тональности. DL-based

Принцип работы RNN:

На каждом шаге сеть получает: вход x_t — текущее слово (или его вектор), скрытое состояние h_{t-1} — память из предыдущего шага, и вычисляет новое состояние:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

h_t — текущее “воспоминание” сети

f — функция активации ($\tanh$, ReLU)

W_x , W_h — веса для входа и памяти

Каждое слово передаёт часть информации о контексте дальше. К концу предложения сеть уже помнит, что было раньше, и использует это при классификации.

Методы анализа тональности. DL-based

LSTM — Long Short-Term Memory (Долгая краткосрочная память)

Идея:

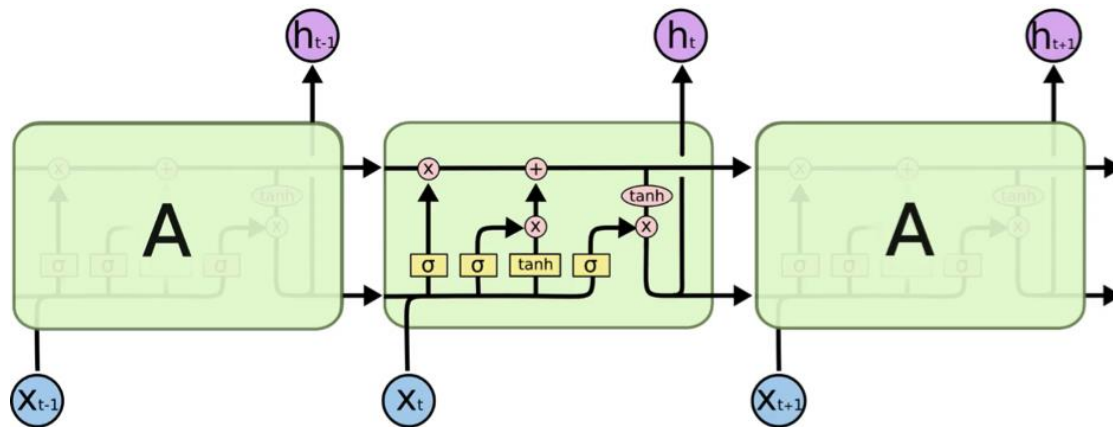
LSTM — это усовершенствованная версия RNN, которая умеет долго помнить контекст и не теряет важную информацию, даже если между словами большая дистанция

Она решает главную проблему RNN — затухание градиента, когда сеть забывает начало длинного предложения

LSTM вводит механизм памяти (cell state) и вентили (gates), которые контролируют, что запомнить, а что забыть

Методы анализа тональности. DL-based

- Линия, проходящая через всю ячейку (от C_{t-1} к C_t) — это “долгосрочная память”, позволяющая сохранять контекст через десятки шагов
- Вентиль забывания (σ) регулирует, что выкинуть
- Входной вентиль ($\sigma + \tanh$) решает, что добавить
- Выходной вентиль (σ) контролирует, что отправить дальше в h_t



Методы анализа тональности. DL-based

Преимущества LSTM:

- Учитывает порядок и контекст слов
- Помнит длинные зависимости
- Хорошо работает на последовательных данных

Недостатки:

- Медленное обучение— обрабатывает слова последовательно, без параллелизма
- Сложность настройки и большое число параметров
- Уступает современным трансформерам (BERT, GPT)

Пример:

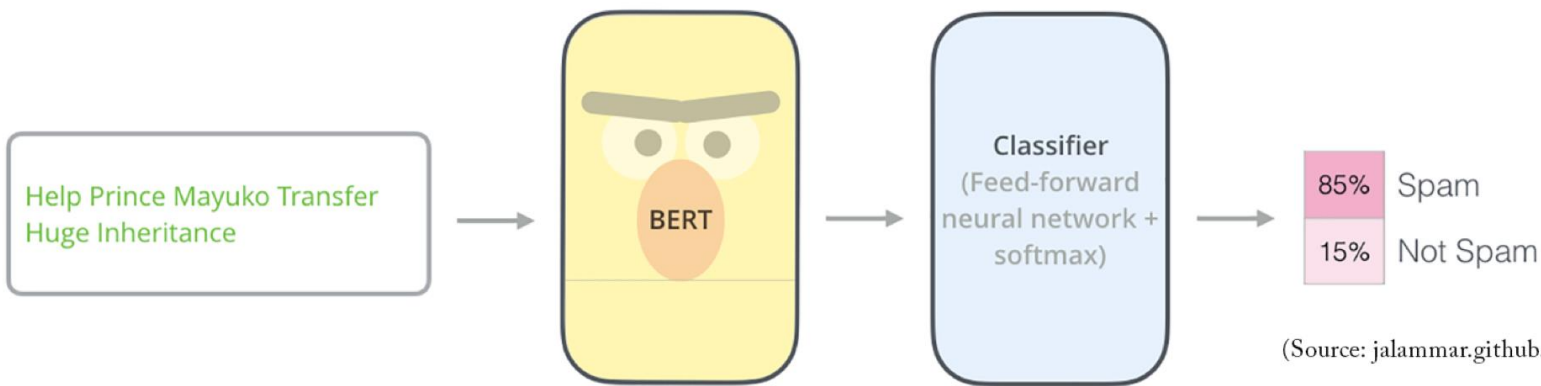
«Фильм, несмотря на медленное начало, оказался великолепным.»

Обычная RNN забудет часть про «великолепным», а LSTM сохранит контекст и поймёт общий позитивный настрой

Методы анализа тональности. DL-based

BERT — Bidirectional Encoder Representations from Transformers

BERT — это модель на основе трансформеров, которая умеет понимать контекст слов с обеих сторон — и слева, и справа. В отличие от RNN/LSTM, которые читают текст последовательно, BERT анализирует все слова одновременно, используя механизм внимания (self-attention)



Методы анализа тональности. DL-based

Как работает:

- Вход: предложение токенизируется (разбивается на части слов)
- Каждый токен превращается в вектор (embedding) с учётом позиции (position embedding)
- Трансформер-энкодер вычисляет, как каждое слово связано со всеми остальными через веса внимания (attention weights)
- Выход: контекстные представления слов, которые отражают их значение в данном предложении

Методы анализа тональности. DL-based

Этапы обучения BERT для анализа тональности

Этап	Что делает	Пример
1. Pre-training	Обучается на огромном корпусе текстов (в т.ч. Википедии) с двумя задачами: Masked Language Model (предсказывает скрытые слова) и Next Sentence Prediction (определяет, связаны ли два предложения)	«I love [MASK]» — «music»
2. Fine-tuning	Дообучается на конкретной задаче, например, анализе тональности, классификации отзывов и т.д.	«Фильм был потрясающий!» — positive

Почему это делается в два этапа

- Языковое понимание (грамматика, контекст) ≠ понимание эмоций
- Предобучение на огромных данных делает модель общей
- Fine-tuning делает её специализированной под задачу

Методы анализа тональности. DL-based

Использование BERT для анализа тональности

Преимущества:

- Двухнаправленное понимание контекста (всё предложение сразу)
- Отлично работает на длинных и сложных текстах
- Можно дообучать под любую задачу (тональность, вопросы, named entities)

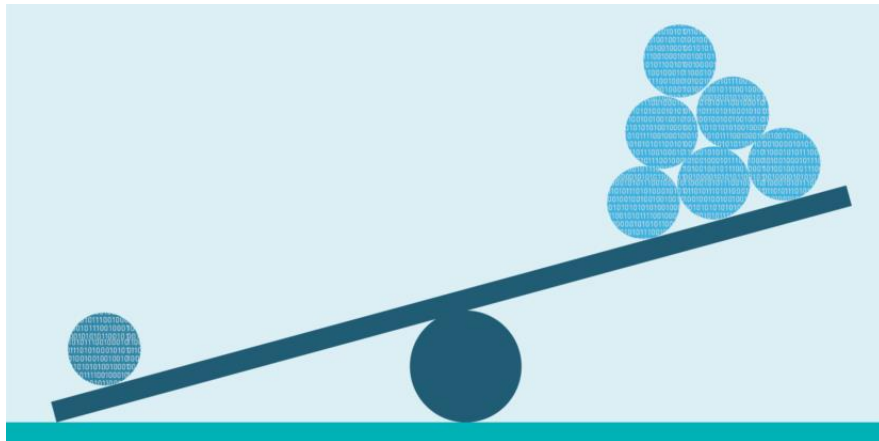
Недостатки:

- Большой размер (сотни миллионов параметров)
- Требуется мощный GPU для обучения
- Медленнее классических моделей (SVM, Logistic Regression)

Метрики качества анализа тональности

Для чего нужны метрики?

После обучения модели важно не только определять правильно тональность, но и понимать, где и как она ошибается — особенно при несбалансированных данных (например, если позитивных отзывов в 10 раз больше, чем негативных)



Метрики качества анализа тональности

Матрица ошибок (Confusion Matrix)

Показывает, где именно модель ошибается при классификации

Каждая строка — реальные классы, каждая колонка — предсказания модели.

фактический отрицательный класс	TN	FP
фактический положительный класс	FN	TP
	спрогнозированный отрицательный класс	спрогнозированный положительный класс

Ошибка	Значение
FP (False Positive)	Модель определила положительный класс, а он на самом деле негативный
FN (False Negative)	Модель не распознала положительный класс
TP, TN (True Positive, True Negative)	Правильные предсказания

Метрики качества анализа тональности

Основные метрики

Метрика	Формула	Что показывает
Accuracy	$((TP + TN) / (TP + TN + FP + FN))$	Доля правильно классифицированных примеров
Precision (точность)	$(TP / (TP + FP))$	Насколько предсказанные положительные классы действительно положительные
Recall (полнота)	$(TP / (TP + FN))$	Какую долю реальных положительных классов модель нашла
F1-score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Баланс между точностью и полнотой

Метрики качества анализа тональности

Пример расчета

Confusion matrix	Predicted Positive	Predicted Negative
Actual Positive	50 (TP)	10 (FN)
Actual Negative	5 (FP)	35 (TN)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{50 + 35}{50 + 35 + 5 + 10} = \frac{85}{100} = 0.85$$

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{50}{50 + 5} = \frac{50}{55} = 0.909$$

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{50}{50 + 10} = \frac{50}{60} = 0.833$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$F1 = 2 \times \frac{0.909 \times 0.833}{0.909 + 0.833} = 2 \times \frac{0.758}{1.742} = 0.870$$

Метрики качества анализа тональности

Проблема несбалансированных классов

Если 90% отзывов — позитивные, то модель, которая всегда выдает положительный класс, получит Accuracy = 0.9, но на самом деле бесполезна

В таких случаях лучше использовать:

- F1-score
- Macro / Weighted average метрики
- Confusion Matrix для анализа ошибок

Метрики качества анализа тональности

Macro Average (макро-усреднение) усредняет метрику по всем классам (каждый класс имеет одинаковый вес)

$$\text{Macro F1} = \frac{1}{N_{classes}} \sum_{i=1}^N F1_i$$

Weighted Average (взвешенное усреднение) усредняет с учётом веса каждого класса (его доли в выборке)

$$\text{Weighted F1} = \frac{\sum_i (F1_i \times n_i)}{\sum_i n_i}$$

В анализе тональности полезно смотреть обе метрики: одна показывает честность по редким эмоциям, другая — реальную точность модели в продакшене

Анализ тональности текста в курсовой работе

Как выполнить анализ тональности для курсовой работы:

- Найти уже размеченный датасет для анализа тональности, наиболее похожий на тот, который нужно спарсить в вашем варианте. Обучить и протестировать несколько моделей на нём. Затем посмотреть и проанализировать, какие результаты выбранная и уже обученная модель выдаёт на примерах из собранного вами датасета
- Самим размечать свой датасет, чтобы максимально повысить точность анализа (рекомендуется только в качестве дополнительной работы для повышения качества результатов)
- Использовать инструменты автоматической разметки на своём датасете (трансформеры с HuggingFace, VADER, TextBlob, Dostoevsky)